CSE111

Data Structures

Lecture 1: Course Introduction

Dr Mervat Mikhail

Introducing the Instructors

Dr. Mervat Mikhail

Computer Science Department CSD

Assistant Professor of Engineering Math Department

Faculty of Engineering, Alexandria University

Email: mmekhaeil@aiu.edu.eg

Course content

Lecture 01: **Introduction - Big O notation**
Lecture 02:**Array & linked list**
Lecture 03:**Stack & queue**
Lecture 04:**Trees**
Lecture 05: **Binary search trees**
Lecture 06: **Hash tables**
Lecture 07: **Heaps**
Lecture 08: **Sorting Techniques**
Lecture 09: **Sorting Techniques (cont.) + Searching**
Lecture 10: **Graph BFS+DFS**
Lecture 11: MST

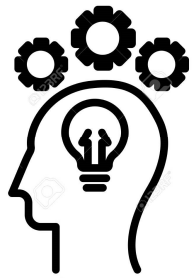
Course Learning Outcomes

1. Demonstrate intermediate knowledge of different types of linked lists, queues, stacks, and priority queues.

2. Demonstrate intermediate knowledge of trees, BST, AVL trees, B-trees family and red-black trees.

3. Demonstrate intermediate knowledge of advanced searching and sorting algorithms.

4. Demonstrate basic knowledge of graphs and graph traverse.



lab work
0 absent
1 attend, no Soln
2 attend, Soln
-2 cheating

Course Assessment

Assessment	Mark Weight
Lab Work	10
Lab Quizzes	10
Lab Project	10
Midterm	20
Extra Opportunities	
Extra lab Project	upto 3
Extra Lec Particip	upto 5
Final	50
Total	100

YW (50)

Extra

Dr Mervat Mikhail

Resources

- Lecture slides and notes
- Lecture YouTube Playlist
- Textbooks (Many!!)
- And More Online Materials

Communication Channels:

- MS Team

Team Code jywrwd7

Introduction to Data Structures

Dr Mervat Mikhail

What is Data Structure

- Programs consists of two things : Algorithms and data structures
- An Algorithm is a step-by-step recipe for solving an instance of problem
- A data Structure represents the logical relationship that exists between individual elements of data to carry out certain task
- A data structures define a way of organizing all data items that consider not only the elements stored but also the relationships between the elements

Classification of data structures

1. **Primitive DS** Ex. int, float, char ⇒ H.W.
2. **Non-Primitive DS**
   A. **Linear** Ex. Array, linked list, stack, queue
   B. **Non-Linear** Ex. Trees, Graphs

Pseudo Code

Ex. write a program that compute the area of the circle

Algo

1- input radius

2- Compute Area = πr^2

3- output

DS

double radius

double Area

Const pi = 3.14

= 22/7

Which DS is more suitable to use when you need

- **Direct access** ⇒ Array A[9] 10th element
- **Sequential access** ⇒ Linked list
- **Reverse access** ⇒ stack

An Algorithm is

1. *Independent of Programming language*

2. *Applicable to inputs of all size and types*

Algorithm Performance

Two areas are important for performance:

***Space efficiency* -** the memory required, also called, space complexity

***Time efficiency* -** the time required, also called time complexity

Space efficiency

Components of space/memory use:

1. Instruction space

Affected by: the compiler, compiler options, target computer (CPU)

2. Data space

Affected by: the data size/dynamically allocated memory, static program variables

3. Run-time stack space

Affected by: the compiler, run-time function calls and recursion, local variables, parameters

Time efficiency

The actual running time depends on many factors:

1. The speed of the computer: CPU.
2. The compiler, compiler options .
3. The quantity of data - ex. search a long list or short.
4. The actual data - ex. in the sequential search if the name is first or last.

5. Algorithm used

Time complexity

Worst case the longest time that an algorithm will use to solve a given problem

Average case the average time that an algorithm will use to solve a given problem

Best Case the shortest time that an algorithm will use to solve a given problem

$O(n)$ $O(n^2)$

Asymptotic Notation

How we measure the complexity ?

$O, \Theta, \omega, o, \Theta, \Omega$

Dr Mervat Mikhail

Asymptotic Complexity

- The notations we use to describe the asymptotic running time of an algorithm are defined in terms of functions whose domains are the set of natural numbers N={0,1,2,3,…….}

When we use asymptotic notation to apply to the running time of an algorithm, we need to understand which running time we mean. Sometimes we are interested in the worst-case running time. Often, however, we wish to characterize the running time no matter what the input.

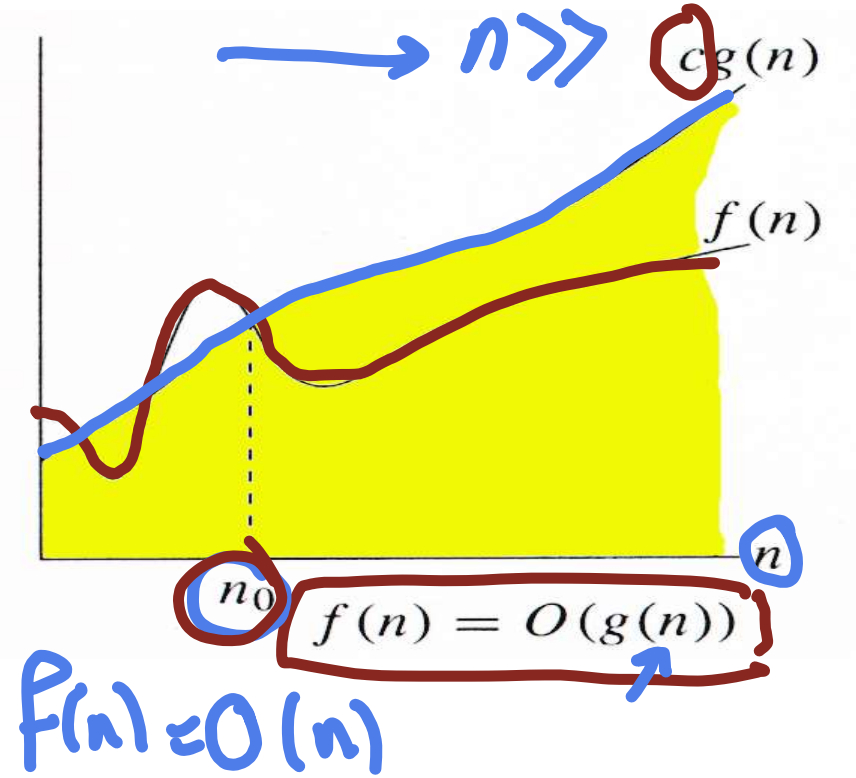
as $n \to \infty$

Big O-notation (Asymptotic upper bound)

Given two nonnegative functions *f* and *g*, ***f = O(g)*** if and only if *g* asymptotically dominates *f*

$O(g(n))$ = {$f(n)$:
∃ positive constants *c* and n_0, such that $\forall n \geq n_0$,
we have $0 \leq f(n) \leq cg(n)$ }

$g(n)$ is an *asymptotic upper bound* for $f(n)$.



$f(n) = O(g(n))$ iff

running time

$g(n)$ is upper bound for $f(n)$

when $n > n_0$

there exist $\exists$ +ve Constant c, n_0 such that

$0 \leq f(n) \leq c\, g(n)$

Examples

1. Is $4n^2+16n+2 = O(n^4)$? yes
2. Is $an+b = O(n^2)$? For any a,b ? yes
3. Is $3n^3 = O(n^4)$?
4. Is $3n^2+25 = O(n^2)$? yes
5. Is $2^{n-1} = O(2^n)$?
6. Is $2^{2n} = O(2^n)$?

$n^4 > 4n^2+16n+2$ for $n>7$

$n^2 > an+b$ as $n>b$

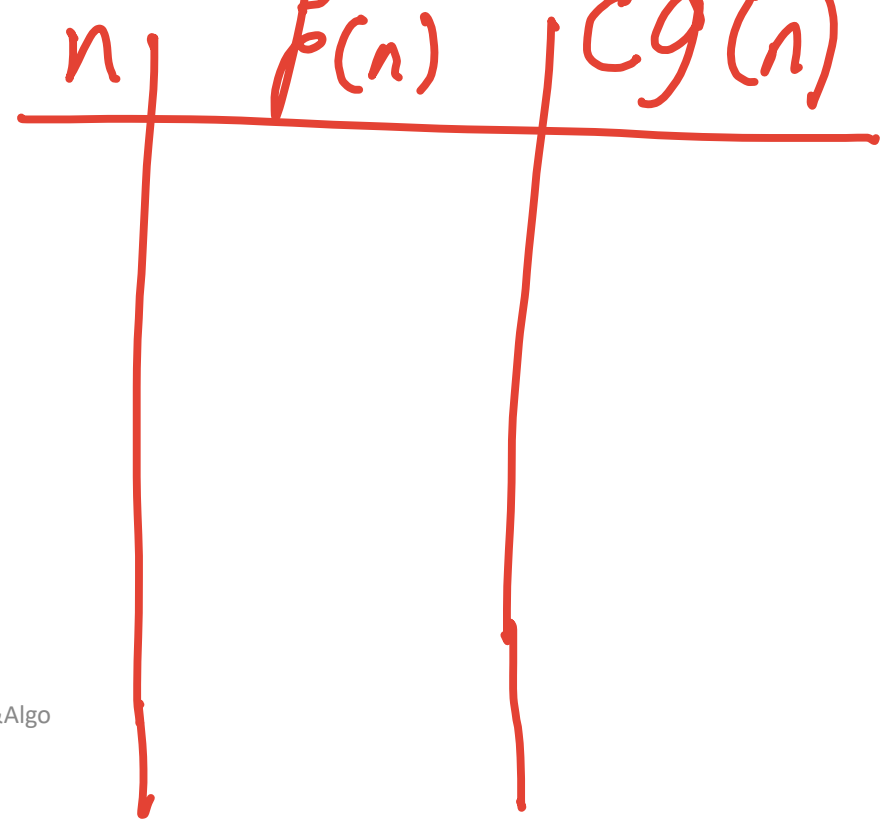
$cn^2 > 3n^2+25$ for $c>3$

Is $4n^2+16n+2 = O(n^4)$?

$\exists\ c, n_0$ such that

$4n^2+16n+2 < cn^4$

put $c=1$



2. IS $an+b = O(n^2)$ for any a,b?

yes

$c = 1$

$an+b < c\,n^2$

for $n \to \infty$

3. Is $3n^3 = O(n^4)$?

yes

put $c = 1$

$3n^3 < n^4$

$n > ?$

4. $3n^2+25 = O(n^2)$

Yes

put $c=4$

$3n^2+25 \leq 4n^2$

$c \geq 3$

5. Is $2^{n-1} = O(2^n)$

Yes

put $c = 1$

$2^{n-1} < c\,2^n$ $\quad c > \frac{1}{2}$

$2^{n-1} < c\,2^n$

6. Is $2^{2n} = O(2^n)$??

$2^{2n} \leq c\,2^n$

$(2^2)^n \leq c\,2^n$

$4^n \leq c\,2^n$

$c \geq \frac{4^n}{2^n}$

No

$c \geq 2^n$

Summary

Assume a Computer Program,

n: input size (Problem size)

$$T(n) = 4n^2 + 16n + 2$$

We can Say that

$T(n)$ is $O(n^2)$

We prefer to use the lowest upperbound

$T(n)$ is also $O(n^3), O(n^4), \ldots$

Important Note

$n \to \infty$

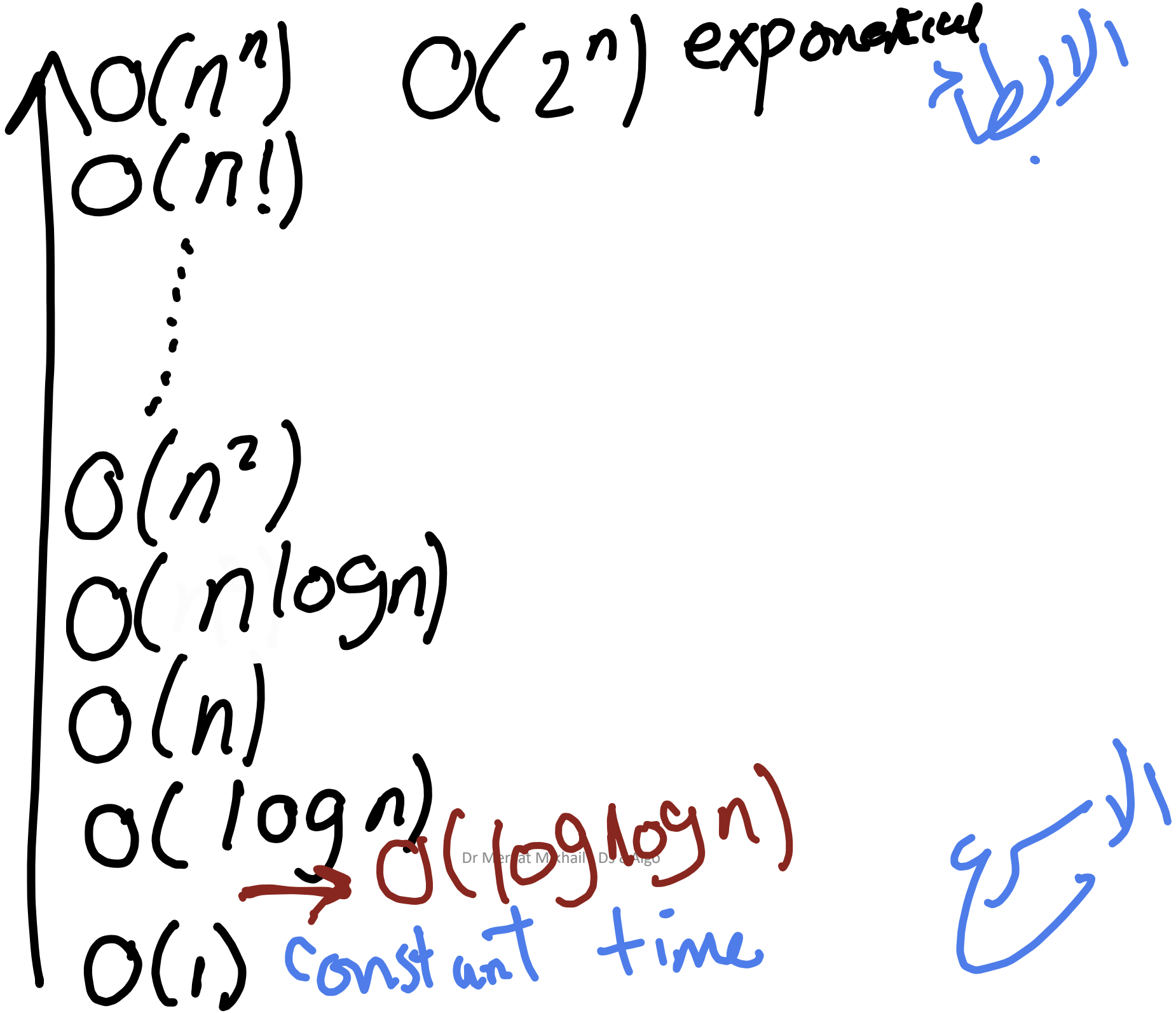
As $n >> $

$$n < n^2 < n^3 < n^4$$

$$n > \log n$$

$$n < n \log n$$

$O(n \log n)$ is more efficient than $O(n^2)$


O(n^n)
O(2^n) exponential
الابطأ
O(n!)
O(n^2)
O(n log n)
O(n)
O(log n)
O(1)
O(log log n)
constant time
الاسرع